# Extending <charconv> support to more character types

**Document number:**
P3876R2

**Date:**
2026-05-11

**Audience:**
SG16

**Project:**
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

**Reply-to:**
Jan Schultke <janschultke@gmail.com>

**Co-authors:**
Peter Bindels <dascandy@gmail.com>

**GitHub Issue:**
wg21.link/P3876/github

**Source:**
github.com/eisenwave/cpp-proposals/blob/master/src/charconv-ext.cow

---

ABSTRACT: `std::to_chars` and `std::from_chars` currently only provide support for `char`, which causes several usability problems. This paper proposes to extend support to all other character types, with essentially the same behavior.

## § Contents

# 1. Revision history

## 1.1. Changes since R1

- rebased wording on [N5032] with the changes in [LWG4522] applied

## 1.2. Changes since R0

SG16 reviewed R0 during several meetings in Q1 2026. Changes to `from_chars` and `to_chars`, as well as support for each character type was polled separately, with (sometimes weak) consensus in favor of the paper. Consequently, only some minor improvements were made:

- expanded abstract
- considered floating-point overloads when counting the size of the `to_chars` overload set in §3.5. Function signature and result types
- fixed wrong claim that libc++ only supports ASCII
- fixed hallucinated `from_string` in example (should have been `from_chars`)
- filed [LWG4522] and removed updates to [diff.cpp26.format]

- changed §3.4. Re-specifying `std::format(std::wformat_string)` from "fixing" `std::format` to merely "re-specifying" it
- explained in § [charconv.from.chars] why using "code unit" is correct (and effectively the status quo)
- rebased §6. Wording on [N5032]

## § 2. Introduction

Support for `char8_t` and other Unicode character types in `std::to_chars` and `std::from_chars` is clearly useful. File formats such as JSON require the use of Unicode character encodings, so an application that deals with JSON may want to use `char8_t` in its APIs and internally. However, when attempting to use `char8_t` for this purpose, one quickly runs into problems:

```
void append_json_number(std::vector<char8_t>& out, int x) {
    // what do I do?
}
```

The user could use the `std::to_chars(char*, char*, int, int)` overload and then transcode to UTF-8 as `char8_t`, but the standard library provides no transcoding facilities yet. Even if there was support, using `char` is an unnecessary middle man.

In general, `std::to_chars` and `std::from_chars` are important cornerstones upon which other facilities are built, or could be built in the future. The lack of support for `char8_t` (and other character types) severely limits what can be done elsewhere:

- `std::to_chars` accepting `char8_t` is arguably a prerequisite for `std::format` with `char8_t` format strings because conversions of arithmetic types are specified in terms of `std::to_chars`.
- `std::print(u8"")` would similarly need `std::to_chars` to function with `char8_t`.
- A hypothetical `std::u8to_string` could not easily be created because `std::to_string` is specified to return `std::format("{}", val)`, `i.e. in terms of std::to_chars.`
- A hypothetical string parsing counterpart to `std::format` would presumably be specified in terms of `std::from_chars`, but this would be problematic if parsing `char8_t` strings is to be supported.

Providing support for Unicode character types would be relatively simple. All characters produced by `std::to_chars` and all characters accepted by `std::from_chars` fall into the Basic Latin (ASCII) block and are part of the basic character set ([lex.charset]). This means that any existing implementation for ASCII-encoded `char` could be made to work with Unicode characters trivially.

## § 3. Design

The design strategy is to prioritize simplicity and performance. `std::from_chars` and `std::to_chars` are meant to be low-level, high-performance conversion functions. Decoding non-ASCII representations of digits, handling UTF-8 encoding errors in detail, etc. are out of the question.

Most of the design choices are obvious, but unfortunately, `<charconv>` functions have been designed as non-templates, which we cannot reasonably perpetuate. Most of the difficult design choices revolve around how to add the new overloads without breaking changes to code which uses `std::to_chars`.

## 3.1. Which character types to support

All character types should be supported by `std::to_chars` and `std::from_chars`. Find rationale for each type below.

### 3.1.1. `char8_t`

Due to how common UTF-8 is and due to `char8_t` now regularly being used to represent UTF-8 text in C++ software, the motivation in §2. Introduction mostly refers to `char8_t`. In fact, there is a dedicated [SG16-Issue] for `char8_t`.

### 3.1.2. `char16_t` and `char32_t`

However, other Unicode encodings such as UTF-16 and UTF-32 are regularly used as well, and if support for UTF-8 exists, it is trivial to support these other encodings (through `char16_t` and `char32_t`) because the conversion functions only deal with code points in the Basic Latin block anyway, where code units are interchangeable.

Overall, the goal should be for a `std::to_chars` implementation to emit the same code units/points for any Unicode character type, and for `std::from_chars` to consume the same code units/points.

### 3.1.3. `wchar_t`

`wchar_t` support is slightly less motivated, and `wchar_t` isn't used much outside of Windows environments. However, it is not difficult to provide support for `wchar_t`, and Windows C++ software may benefit from this support (e.g. when feeding the output of `std::to_chars` into Windows API functions accepting `LPCWSTR` (`const wchar_t*`)).

## 3.2. `to_chars`

The output format of `to_chars` should be identical to that for `char`. This is easily implementable because all characters produced by `to_chars` are Basic Latin characters in the basic character set.

## 3.3. `from_chars`

The formats accepted by `from_chars` should be identical to those for `char`, which are specified in terms of functions like `strtol` in the `"C"` locale.

`from_chars` for Unicode characters should not accept any further constructs such as parsing `u8" IV "` as `4` because this goes against its stated design goal of being a low-level, high-performance utility for parsing numbers.

### 3.3.1. Unicode error handling

It is possible that a user attempts to invoke `std::from_chars` on a malformed Unicode string. However, this does not mean that any special consideration to UTF-8 or other encodings needs to be paid. `std::from_chars` simply assumes that the given character range contains a pattern (for integers, a sequence of digits with optional `'-'` prefix) at the start of the range; this pattern is made entirely of characters in the Basic Latin block.

EXAMPLE: The following code demonstrates the intended behavior:

```
string_view cstr = "123z"; // OK, not malformed
int i1;
const auto [p1, e1] = from_chars(cstr.data(), cstr.data() + cstr.size(),

u8string_view u8str = u8"123\xFF"; // malformed UTF-8
int i2;
const auto [p2, e2] = from_chars(u8str.data(), u8str.data() + u8str.size(

assert(i1 == i2);                                // holds, both i1 and i2 equal 1
assert(p1 - cstr.data() == p2 - u8str.data()); // holds, both patterns are three
```

All Unicode encodings are designed so that code *only* code points in the Basic Latin block can be encoded with code units in the range [`0`, `0x7f`). This means that simply treating greater code units as not part of the `std::from_chars` pattern (which any implementation for ASCII-based `char` does already) is a proper way of Unicode error handling.

## 3.4. Re-specifying `std::format(std::wformat_string)`

Since this proposal argues for `wchar_t` support in `std::to_chars`, it makes sense to re-specify `std::format` to call `std::to_chars` "directly". The current wording following [LWG4522] being merged into the working draft specifies transcoding to the wide literal encoding.

Once `std::to_chars` supports `wchar_t`, `std::format` should simply call that overload directly, without transcoding. Assuming [LWG4522] is accepted and transcoding already happens, this effectively produces the same result but with simpler wording.

## 3.5. Function signature and result types

`to_chars` and `from_chars` are not function templates despite working with a wide variety of arithmetic types. At least, there need to be 11 overloads for each signed and unsigned integer

type and for `char`, as well as up to 3×3 overloads for standard floating-point types. There are at least 20 `to_chars` overloads mandated by the standard, though recently added support for `__int128` and `unsigned __int128` pushed that number to 22 for GCC and Clang.

If we also added a non-template overload for each character type, this would result in an absurd overload set of 110 functions (`{char, wchar_t, char8_t, char16_t, char32_t}` × `{char, signed char, ..., long double}`). Such a huge overload set is clearly undesirable, so function templates are necessary.

> NOTE: 110 is also merely a lower bound. There could easily be 200 or 300 `to_chars` overloads once `<stdfloat>` types and other extended arithmetic types are factored in.

### 3.5.1. Result type

The existing `std::to_chars_result` and `std::from_chars_result` classes cannot be turned into class templates without breaking both API and ABI. That is because any existing aliases or uses of these types in function parameters, return types, etc. would break if they were turned into templates. Name mangling would also change.

There is also no good name for a new class template, and if that was used for Unicode characters, the asymmetry with the `char` overloads would be even more apparent. However, we could create one result type per character, as well as alias templates `std::to_chars_result_t` and `std::from_chars_result_t` which select the appropriate result class.

Another possible option is to create a base class as follows:

```
template<class T>
struct basic_to_chars_result { /* ... */ };

struct to_chars_result : basic_to_chars_result<char> { };
using u8to_chars_result = basic_to_chars_result<char8_t> { }; // maybe?
// ...
```

This would also allow deduction of `T` from `basic_to_chars_result`, unlike adding a new set of independent types. However, this also technically breaks API because moving members into a base class changes aggregate initialization.

Overall, the safest option is to make no changes to the existing result types.

### 3.5.2. Summary

In code, the design can be summarized as follows:

```
template<class T>
  concept character-type = any character type;

struct to_chars_result {
  char* ptr;
  errc ec;
```

```
  friend bool operator==(const to_chars_result&, const to_chars_result&) = defa
  constexpr explicit operator bool() const noexcept { return ec == errc{}; }
};
// analogous classes:
struct wto_chars_result { /* ... */ };
struct u8to_chars_result { /* ... */ };
struct u16to_chars_result { /* ... */ };
struct u32to_chars_result { /* ... */ };

template<character-type T>
  using to_chars_result_t = one of the result types above;

// pre-existing overload:
constexpr to_chars_result
  to_chars(char* first, char* last, integer-type value, int base = 10);
// new function template:
template<character-type T, integer-type-concept U>
  constexpr to_chars_result_t<T> to_chars(T* first, T* last, U value, int base

// same approach for floating-point types:
to_chars_result to_chars(char* first, char* last,
                         floating-point-type value);
template<character-type T, floating-point-type-concept U>
  to_chars_result_t<T> to_chars(T* first, T* last, U value);

to_chars_result to_chars(char* first, char* last,
                         floating-point-type value, chars_format fmt);
template<character-type T, floating-point-type-concept U>
  to_chars_result_t<T> to_chars(T* first, T* last, U value, chars_format fmt);

to_chars_result to_chars(char* first, char* last,
                         floating-point-type value, chars_format fmt, int preci
template<character-type T, floating-point-type-concept U>
  to_chars_result_t<T> to_chars(T* first, T* last, U value, chars_format fmt, i
```

```
struct from_chars_result {
  const char* ptr;
  errc ec;
  friend bool operator==(const from_chars_result&, const from_chars_result&) =
  constexpr explicit operator bool() const noexcept { return ec == errc{}; }
};
// analogous classes:
struct wfrom_chars_result { /* ... */ };
struct u8from_chars_result { /* ... */ };
struct u16from_chars_result { /* ... */ };
struct u32from_chars_result { /* ... */ };

template<character-type T>
  using from_chars_result_t = one of the result types above;

// pre-existing overload:
constexpr from_chars_result
  from_chars(const char* first, const char* last, integer-type& value, int base
```

```
// new function template:
template<character-type T, integer-type-concept U>
  constexpr from_chars_result_t<T> from_chars(T* first, T* last, U& value, int

// same approach for floating-point types:
from_chars_result from_chars(const char* first, const char* last,
                             floating-point-type& value,
                             chars_format fmt = chars_format::general);
template<character-type T, integer-type-concept U>
  from_chars_result_t<T> from_chars(T* first, T* last, U& value,
                                    chars_format fmt = chars_format::general);
```

## 3.6. `constexpr` floating-point overloads

If [P3652R1] "Constexpr floating-point <charconv> functions" is accepted, all new templated overloads should be made `constexpr`. There is no good reason why only the `char` overload should be `constexpr`.

> NOTE: A possible implementation is to call the `constexpr to_chars(char*, /* .../*)` overload and to transcode from the ordinary literal encoding to the desired encoding.

## 3.7. More composable interface taking `std::span` or `std::string_view`

It is worth noting that there is a stale proposal [P2584R0] "A More Composable `from_chars`" which proposes additional overloads taking `std::span`, superseding the even more stale [P2007R0] "`std::from_chars` should work with `std::string_view`".

Such changes are orthogonal to what is proposed here. However it needs to be considered what impact such new overloads would have on the functions added here. In particular, [P2584R0] proposes an interface such as:

```
template<class T>
  constexpr from_chars_result_range<T> from_chars(span<const char> rng, int bas
```

If this was added, a non-breaking change would require adding four more overloads taking `span<char8_t>`, `span<char16_t>`, `span<char32_t>`, and `span<wchar_t>`. A similar change to `to_chars` would actually expand the overload set by 20 function templates (5 character types × (1 integer overload + 3 floating-point overloads)), resulting in 11 + 4 + 20 = 35 candidates in the `to_chars` overload set (including the ones proposed here).

With the benefit of foresight, perhaps we should aim at a smaller overload set and take a `R&&` range parameter instead. In any case, those changes are not within the scope of this proposal.

# 4. Impact on existing code

The proposal is a pure extension of the `std::to_chars` and `std::from_chars` overload sets. The existing non-template overloads for `char` and various arithmetic types are preserved.

# 5. Implementation experience

Any existing implementation of `std::to_chars` and `std::from_chars` for a platform with ASCII-based `char` (Windows, POSIX, etc.) is *numerically* implementing what is proposed here. That is, the implementation may not use `char8_t`, but it produces or consumes `char` values with the same numeric values.

## 5.1. Implementation survey

Find below a summary of existing implementations of `to_chars` in the three major standard libraries. This is necessary to understand what difficulties implementations would face when supporting additional character types.

| Functions | libstdc++ | libc++ | MSVC STL |
|---|---|---|---|
| `to_chars` (integer) | std/charconv | to_chars_integral.h | inc/charconv |
| `to_chars` (floating-point) | floating_to_chars.cc | to_chars_floating_point.h | inc/charconv |
| `from_chars` (integer) | std/charconv | to_chars_integral.h | inc/charconv |
| `from_chars` (floating-point) | floating_from_chars.cc | from_chars_floating_point.h | inc/charconv |

All implementations are quite similar: the underlying function performing the conversion is a function template with type parameter `T`, to handle integer types or floating-point types in bulk. These could easily be turned into templates which also have a `charT` type parameter. The only difficulty would be converting the existing uses of ordinary character and string literals into correctly typed literals for `charT`.

EXAMPLE: libc++ contains the following line of code for inserting a plus sign into the exponent in `to_chars`:

```
*_First++ = '+';
```

This may have to be converted into `static_cast<charT>('+')` to avoid implicit conversion warnings, but that is only correct in an ASCII-only implementation.

libc++ supports EBCDIC and is used on z/OS (see https://reviews.llvm.org/D114813). Another library with EBCDIC support is the IBM XL C++ for z/OS, but according to IBM's documentation, no `<charconv>` implementation exists yet. Even if `char` is non-ASCII, the "fix" is not much harder than a `static_cast`:

EXAMPLE: The libc++ snippet could have also been fixed in an EBCDIC-compatible way:

```
// Implement this once, and use it anywhere necessary:
template<class _T>
constexpr _T _Encode(char32_t code_point) {
  if constexpr (^^_T == ^^char8_t || ^^_T == ^^char16_t || ^^_T == ^^cha
    return _T(code_point);
  } else {
    return _Encode_ebcdic(code_point);
  }
}

// Now, assuming we are in a function template with _CharT type paramete
*_First++ = _Encode<_CharT>(U'+');
```

## 5.2. New alias templates

The proposed alias templates can be implemented as follows:

```
template<class T>
concept __character_type =
  ^^T == ^^char || ^^T == ^^char8_t || ^^T == ^^char16_t || ^^T == ^^char32_t |

template<__character_type T>
using to_chars_result_t = [:
    ^^T == ^^char     ? ^^to_chars_result
  : ^^T == ^^char8_t  ? ^^u8to_chars_result
  : ^^T == ^^char16_t ? ^^u16to_chars_result
  : ^^T == ^^char32_t ? ^^u32to_chars_result
                      : ^^wto_chars_result
:];
```

The implementation of `from_chars_result_t` is analogous.

> NOTE: The implementation doesn't actually require C++26 reflection. More traditional alternatives like `std::conditional_t` also work.

# 6. Wording

The following changes are relative to [N5032] with the changes in [LWG4522] applied.

## [version.syn]

In [version.syn], bump the feature-test macro:

```
#define __cpp_lib_to_chars ~~202306L~~ 20XXXXL // also in <charconv>
```

DRAFTING NOTE: The `__cpp_lib_constexpr_charconv` and `__cpp_lib_freestanding_charconv` macros are not bumped.

## § [charconv.syn]

In [charconv.syn], modify the synopsis as follows:

```
namespace std {
  // exposition-only concepts
  template<class T>
    concept character-type = see below;

  // floating-point format for primitive numerical conversion
  enum class chars_format {
    scientific = unspecified,
    fixed = unspecified,
    hex = unspecified,
    general = fixed | scientific
  };

  // [charconv.to.chars], primitive numerical output conversion
  struct to_chars_result {
    char* ptr;
    errc ec;
    friend bool operator==(const to_chars_result&, const to_chars_result&)
    constexpr explicit operator bool() const noexcept { return ec == errc
  };
  struct u8to_chars_result {
    char8_t* ptr;
    errc ec;
    friend bool operator==(const u8to_chars_result&, const u8to_chars_resu
    constexpr explicit operator bool() const noexcept { return ec == errc
  };
  struct u16to_chars_result {
    char16_t* ptr;
    errc ec;
    friend bool operator==(const u16to_chars_result&, const u16to_chars_re
    constexpr explicit operator bool() const noexcept { return ec == errc
  };
  struct u32to_chars_result {
    char32_t* ptr;
    errc ec;
    friend bool operator==(const u32to_chars_result&, const u32to_chars_re
    constexpr explicit operator bool() const noexcept { return ec == errc
  };
  struct wto_chars_result {
    wchar_t* ptr;
    errc ec;
    friend bool operator==(const wto_chars_result&, const wto_chars_result
    constexpr explicit operator bool() const noexcept { return ec == errc
  };
```

```
template<class T>                                                    //
  using to_chars_result_t = see below;

constexpr to_chars_result to_chars(char* first, char* last,          //
                                   integer-type value, int base = 10);
template<class charT, class V>                                       //
  constexpr to_chars_result_t<charT> to_chars(charT* first, charT* last,
                                              V value, int base = 10);
to_chars_result to_chars(char* first, char* last,                    //
                         bool value, int base = 10) = delete;

to_chars_result to_chars(char* first, char* last,                    //
                         floating-point-type value);
template<class charT, class V>                                       //
  to_chars_result_t<charT> to_chars(charT* first, charT* last, V value);
to_chars_result to_chars(char* first, char* last,                    //
                         floating-point-type value, chars_format fmt);
template<class charT, class V>                                       //
  to_chars_result_t<charT> to_chars(charT* first, charT* last, V value,
                                    chars_format fmt);
to_chars_result to_chars(char* first, char* last,                    //
                         floating-point-type value, chars_format fmt, in
template<class charT, class V>                                       //
  to_chars_result_t<charT> to_chars(charT* first, charT* last, V value,
                                    chars_format fmt, int precision);

// [charconv.from.chars], primitive numerical input conversion
struct from_chars_result {                                           //
  const char* ptr;
  errc ec;
  friend bool operator==(const from_chars_result&, const from_chars_resu
  constexpr explicit operator bool() const noexcept { return ec == errc
};
struct u8from_chars_result {                                         //
  const char8_t* ptr;
  errc ec;
  friend bool operator==(const u8from_chars_result&, const u8from_chars_
  constexpr explicit operator bool() const noexcept { return ec == errc
};
struct u16from_chars_result {                                        //
  const char16_t* ptr;
  errc ec;
  friend bool operator==(const u16from_chars_result&, const u16from_char
  constexpr explicit operator bool() const noexcept { return ec == errc
};
struct u32from_chars_result {                                        //
  const char32_t* ptr;
  errc ec;
  friend bool operator==(const u32from_chars_result&, const u32from_char
  constexpr explicit operator bool() const noexcept { return ec == errc
};
struct wfrom_chars_result {                                          //
```

```
    const wchar_t* ptr;
    errc ec;
    friend bool operator==(const wfrom_chars_result&, const wfrom_chars_re
    constexpr explicit operator bool() const noexcept { return ec == errc{
  };

  template<class T>                                                    //
    using from_chars_result_t = see below;

  constexpr from_chars_result from_chars(const char* first, const char* la
                                         integer-type& value, int base = 1
  template<class charT, class V>
    constexpr from_chars_result_t<charT> from_chars(const charT* first, co
                                                    V& value, int base = 1

  from_chars_result from_chars(const char* first, const char* last,     //
                               floating-point-type& value,
                               chars_format fmt = chars_format::general);
  template<class charT, class V>                                       //
    from_chars_result_t<charT> from_chars(const charT* first, const charT*
                                          V& value,
                                          chars_format fmt = chars_format:
}
```

Immediately preceding [charconv.syn] paragraph 2, insert a paragraph as follows:

The exposition-only concept *character-type* is modeled by any character type ([basic.fundamental]).

## § [charconv.to.chars]

Immediately following [charconv.to.chars] paragraph 1, insert the following paragraph:

The output style of all functions named `to_chars` is specified in terms of characters in the basic character set (and thus in terms of their Unicode code points) or directly in terms of code points. The output code points are inserted into the range [`first`, `last`) by encoding them in the respective literal encoding for character literals of the type of `*first`.

Immediately following [charconv.to.chars] paragraph 3, insert the following item:

```
template<character-type T>
  using to_chars_result_t = see below;
```

*Result*:

- `to_chars_result` if `T` is `char`,
- `u8to_chars_result` if `T` is `char8_t`,
- `u16to_chars_result` if `T` is `char16_t`,

- `u32to_chars_result` if `T` is `char32_t`, and
- `wto_chars_result` if `T` is `wchar_t`.

DRAFTING NOTE: Using *Result*: elements for non-functions may be surprising, but is permitted by [structure.specifications] and has been done in various other places.

Modify the overload for *`integer-type`* as follows:

```
constexpr to_chars_result to_chars(char* first, char* last, integer-type \
template<class charT, class V>
  constexpr to_chars_result_t<charT> to_chars(charT* first, charT* last,
                                              V value, int base = 10);
```

*Constraints*: `charT` is a character type ([basic.fundamental]). `V` is a signed or unsigned integer type or `char`.

*Preconditions*: `base` has a value between 2 and 36 (inclusive).

*Effects*: The value of `value` is converted to a string of digits in the given base (with no redundant leading zeroes). Digits in the range 0..9 are represented as U+0030..U+0039 DIGIT ZERO..NINE, and digits in the range 10..35 ~~(inclusive)~~ are represented as ~~lowercase characters a..z~~ U+0061..U+007A LATIN SMALL LETTER A..Z. If `value` is less than zero, the representation starts with ~~`'-'`~~ U+002D HYPHEN-MINUS.

*Throws*: Nothing.

DRAFTING NOTE: This change has a merge conflict with [LWG4421].

The "(inclusive)" is removed because the range notation "A..B" is universally inclusive, with no disambiguation required. Such range notation is already used in [lex.charset] without any attempt at disambiguation.

Modify the overloads for *`floating-point-type`* as follows:

```
to_chars_result to_chars(char* first, char* last, floating-point-type valu
template<class charT, class V>
  to_chars_result_t<charT> to_chars(charT* first, charT* last, V value);
```

*Constraints*: `charT` is a character type ([basic.fundamental]). `V` is a cv-unqualified floating-point type.

*Effects*: `value` is converted to a string in the style of `printf` in the `"C"` locale. The conversion specifier is `f` or `e`, chosen according to the requirement for a shortest representation (see above); a tie is resolved in favor of `f`.

*Throws*: Nothing.

```
to_chars_result to_chars(char* first, char* last, floating-point-type valu
template<class charT, class V>
  to_chars_result_t<charT> to_chars(charT* first, charT* last, V value, ch
```

*Constraints*: `charT` is a character type ([basic.fundamental]). `V` is a cv-unqualified floating-point type.

*Preconditions*: `fmt` has the value of one of the enumerators of `chars_format`.

*Effects*: `value` is converted to a string in the style of `printf` in the `"C"` locale.

*Throws*: Nothing.

```
to_chars_result to_chars(char* first, char* last, floating-point-type valu
                         chars_format fmt, int precision);
template<class charT, class V>
  to_chars_result_t<charT> to_chars(charT* first, charT* last, V value,
                                    chars_format fmt, int precision);
```

*Constraints*: `charT` is a character type ([basic.fundamental]). `V` is a cv-unqualified floating-point type.

*Preconditions*: `fmt` has the value of one of the enumerators of `chars_format`.

*Effects*: `value` is converted to a string in the style of `printf` in the `"C"` locale with the given precision.

*Throws*: Nothing.

DRAFTING NOTE: See [N3047-fprintf] for C23 wording. To give an example, the output format for `printf` is worded as follows:

> `f,F` — A `double` argument representing a floating-point number is converted to decimal notation in the style *[-]ddd.ddd*, where the number of digits after the decimal-point character is equal to the precision specification.

This abstract description of the output style (where presumably, "-" and "." are intended to represented characters in the basic character set) can be applied to the new overloads working with `char8_t` and other types, just like it could have been applied to `char`.

It may be beneficial to reword the whole subclause in terms of code points and decoupled from C wording, but this would take considerable effort and isn't necessary for this proposal.

If [P3652R1] has been accepted or a later paper marked the existing *`floating-point`* overloads `constexpr`, modify all the added overloads as follows:

```
template<class charT, class V>
  constexpr to_chars_result_t<charT> […]
```

## § [charconv.from.chars]

Modify [charconv.from.chars] paragraph 1 as follows:

> All functions named `from_chars` analyze the string [`first`, `last`) for a pattern, where [`first`, `last`) is required to be a valid range. If no ~~characters~~ **code units** match the pattern, `value` is unmodified, the member `ptr` of the return value is `first` and the member `ec` is equal to `err::invalid_argument`.
>
> [*Note*: If the pattern allows for an optional sign, but the string has no digit ~~characters~~ **code units** following the sign, no ~~characters~~ **code units** match the pattern. — *end note*]
>
> Otherwise, the ~~characters~~ **code units** matching the pattern are interpreted as a representation of a value of the type of `value`. The member `ptr` of the return value points to the first ~~character~~ **code unit** not matching the pattern, or has the value `last` if all ~~characters~~ **code units** match. If the parsed value is not in the range representable by the type of `value`, `value` is unmodified and the member `ec` of the return value is equal to `err::result_out_of_range`. Otherwise, `value` is set to the parsed value, after rounding according to `round_to_nearest` ([round.style]), and the member `ec` is value-initialized.

> DRAFTING NOTE: The current use of "character" in the wording is unclear because it could equally mean that the `from_chars` operates on the decoded code points or on the code units of the string. `from_chars` leans on the "subject sequence" of `strtol` for wording (see C23 §7.24.1.7), which is worded in terms of "characters", presumably referring to "character" as in "single-byte character" (see C23 §3.10.1). This means that `from_chars` is already intended to operate on bytes or code units.

Immediately following [charconv.from.chars] paragraph 1, insert a new paragraph:

> The output style of all functions named `from_chars` is specified in terms of characters in the basic character set (and thus in terms of their Unicode code points) or directly in terms of code points. The analyzed pattern consists of those code points, encoded as code units in the respective literal encoding for character literals of the cv-unqualified type of `*first`.
>
> [*Note*: In either form of specification, the pattern consists of code units encoding characters in the basic character set ([lex.charset]), meaning that each code unit encodes exactly one such character. Illegal code units or code units representing characters outside the basic character set are not handled specially; those code units are simply not part of the pattern.
>
> [*Example*:

```
u8string_view s = u8"123\xFF";       // well-formed string-literal containing malform
int value;
u8from_chars_result r = from_chars(s.data(), s.data() + s.length(), value

assert(value == 123);                // holds
```

```
assert(r.ptr == s.data() + 3);       // holds
assert(r.ec == errc{});              // holds
```

*— end example*] *— end note*]

Immediately following the inserted paragraph, insert the following item:

```
template<character-type T>
  using from_chars_result_t = see below;
```

*Result*:

- `from_chars_result` if `T` is `char`,
- `u8from_chars_result` if `T` is `char8_t`,
- `u16from_chars_result` if `T` is `char16_t`,
- `u32from_chars_result` if `T` is `char32_t`, and
- `wfrom_chars_result` if `T` is `wchar_t`.

Modify the overload for *integer-type* as follows:

```
constexpr from_chars_result from_chars(const char* first, const char* last
                                       integer-type& value, int base = 10)
template<class charT, class V>
  constexpr from_chars_result_t<charT> from_chars(const charT* first, cons
                                                  V& value, int base = 10)
```

*Constraints*: `charT` is a character type ([basic.fundamental]). `V` is a signed or unsigned integer type or `char`.

*Preconditions*: `base` has a value between 2 and 36 (inclusive).

*Effects*: ~~The pattern is the expected form of the subject sequence in the `"C"` locale for the given nonzero base, as described for `strtol`, except that no `"0x"` or `"0X"` prefix shall appear if the value of base is 16, and except that `'-'` is the only sign that may appear, and only if value has a signed type.~~ The pattern is a sequence of digits in the given base, where leading zeroes are ignored. The code points U+0030..U+0039 DIGIT ZERO..NINE represent digits in the range 0..9; Both U+0041..U+005A LATIN CAPITAL LETTER A..Z and U+0061..U+007A LATIN SMALL LETTER A..Z represent digits in the range 10..35. If `value` is of signed type, the pattern starts with an optional U+002D HYPHEN-MINUS which causes the resulting `value` to be negative.

*Throws*: Nothing.

DRAFTING NOTE: By the time this wording is reviewed, [LWG4430] will most likely have been merged, which additionally ignores `"0b"` and `"0B"` base prefixes. This has no impact on the proposed change because the *Effects*: are rewritten from scratch.

It would be possible to keep basing the wording on `strtol`, but this is quite problematic. [LWG4430] already fixed the accidental parsing of `"0b"` prefixes in `from_chars`, and additional changes will be required for `"0o"`, which is added to C2y for octal prefixes.

There are so many deviations from the `strtol` pattern that it arguably provides negative value to specify `to_chars` in terms of it.

Modify the overload for *`floating-point-type`* as follows:

```
from_chars_result from_chars(const char* first, const char* last, floating
                             chars_format fmt = chars_format::general);
template<class charT, class V>
  from_chars_result_t<charT> from_chars(const charT* first, const charT* l
                                        chars_format fmt = chars_format::g
```

*Constraints*: `charT` is a character type ([basic.fundamental]). `V` is a cv-unqualified floating-point type.

*Preconditions*: `fmt` has the value of one of the enumerators of `chars_format`.

*Effects*: The pattern is the expected form of the subject sequence in the `"C"` locale, as described for `strtod`, except that

- ~~the sign `'+'`~~ U+002B PLUS SIGN may only appear in the exponent part;
- if `fmt` has `chars_format::scientific` set but not `chars_format::fixed`, the otherwise optional exponent part shall appear;
- if `fmt` has `chars_format::fixed` set but not `chars_format::scientific`, the optional exponent part shall not appear; and
- if `fmt` is `chars_format::hex`, the prefix ~~`"0x"` or `"0X"`~~ `0x` is assumed to precede the string for the purpose of determining the resulting `value`, but is not part of the pattern.

In any case, the resulting `value` is one of at most two floating-point values closest to the value of the string matching the pattern.

*Throws*: Nothing.

DRAFTING NOTE: This change includes a drive-by fix which supersedes [EDIT6848].

## § [format.string.std]

Modify [format.string.std] paragraph 20 as follows:

The meaning of some non-string presentation types is defined in terms of a call to `to_chars`. In such cases, let [`first`, `last`) be a range of elements of type `charT`, large enough to hold the `to_chars` output, and let `value` be the formatting argument value.

> Formatting is done as if by calling `to_chars` as specified~~, transcoding the `to_chars` output to the wide literal encoding if `charT` is `wchar_t`,~~ and copying the output through the output iterator of the format context.
>
> [*Note*: Additional padding and adjustments are performed prior to copying the output through the output iterator as specified by the format specifiers. — *end note*]

> DRAFTING NOTE: The additional transcoding from the ordinary literal encoding to the wide literal encoding used to be necessary because `to_chars` only supported `char`. With the changes in this paper, `to_chars` can simply produce output with the same character type and encoding as `format`, so additional transcoding becomes unnecessary.

# § 7. References


[N5032] *Thomas Köppe.* Working Draft, Programming Languages — C++ (2025-12-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf

[P2007R0] *Mateusz Pusz.* std::from_chars should work with std::string_view (2020-01-10)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2007r0.html

[P2584R0] *Corentin Jabot.* A More Composable from_chars (2022-05-12)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2584r0.pdf

[P3652R1] *Lénárd Szolnoki.* Constexpr floating-point <charconv> functions (2025-04-16)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3652r1.html

[LWG4421] *Jan Schultke.* Clarify the output encoding of to_chars for integers
https://cplusplus.github.io/LWG/issue4421

[LWG4430] *Jan Schultke.* from_chars should not parse "0b" base prefixes
https://cplusplus.github.io/LWG/issue4430

[LWG4522] *Jan Schultke.* Clarify that std::format transcodes for std::wformat_strings
https://cplusplus.github.io/LWG/issue4522

[SG16-Issue] std::to_chars/std::from_chars overloads for char8_t (#38)
https://github.com/sg16-unicode/sg16/issues/38

[EDIT6848] *Jan Schultke.* [charconv.from.chars] Clarify the role of a 0x prefix in from_chars
https://github.com/cplusplus/draft/pull/6848

[N3047-fprintf] N3047 7.23.6.1 [The fprintf function]
https://www.iso-9899.info/n3047.html#7.23.6.1